

numberAI: Building an Image Classifier from Scratch

Project Documentation

May 26, 2026

Abstract

This document outlines the development, architecture, and findings of the **numberAI** project. The objective of this project is to recreate machine learning functionality from scratch, without relying on pre-trained models or established neural network structures. By utilizing simple matrix multiplication and a trial-and-error training approach, the project aims to categorize number images. Additionally, it details the planned second phase involving quantization and distillation for deployment on resource-constrained embedded systems such as Arduino.

Contents

1	Introduction	3
1.1	Project Motivation	3
1.2	Goals and Phases	3
2	Phase 1: Architecture Iterations	3
2.1	Version 0: Initial Attempt	3
2.1.1	Architecture and Methodology	3
2.1.2	Results and Failures	4
2.1.3	Points to Solve in Next Iteration	4
2.2	Version 1.0: Non-Linearity and Optimization	4
2.2.1	Architecture Upgrades	4
2.2.2	Methodology Upgrades	5
2.2.3	Version 1.0 Training Results	5
2.2.4	Points to Solve in Next Iteration	6
2.3	Version 1.1: Vectorized Blame Assignment (Backpropagation)	7
2.3.1	Methodology Upgrades	7
2.3.2	Version 1.1 Training Results	7
2.3.3	Points to Solve in Next Iteration	8
2.4	Version 1.2: Improved Dataset Generalization	9
2.4.1	Methodology and Results	9
2.4.2	Points to Solve in Next Iteration	10
2.5	Version 1.2.1 & 1.4.1: Scaling to Full Resolution	10
2.6	Version 1.3 & 1.4: Hybrid Pooling Experiment	10
2.7	Version 1.5 & 1.5.1: Deep Neural Network Transition	10
2.8	Version 1.6 & 1.7: The 4-Branch Architecture	10
2.9	Version 1.8: 7-Branch Geometric Slicing	11

2.10	Version 1.8.1: 12-Branch Spatial Expansion	11
2.11	Version 2.0: Introduction of Convolution	11
2.11.1	Architecture Upgrades	11
2.11.2	Version 2.0 Training Results	12
2.11.3	Points to Solve in Next Iteration	12
2.12	Version 2.1: PyTorch Optimization	12
2.12.1	Architecture and Methodology Upgrades	12
2.12.2	Version 2.1 Training Results	13
2.12.3	Points to Solve in Next Iteration	14
2.13	Version 3.0: CFNet (Asymmetrical Input Slicing)	14
2.13.1	Architecture Upgrades (RFNet)	14
2.14	Version 3.1: Decoupled Asymmetrical Factor Net (DAFNet)	15
2.14.1	Architecture Details	15
2.14.2	Training and Results	16
3	Phase 2: Embedded Systems Optimization	16
3.1	Teacher Model: High-Capacity WaveMix Architecture (v4.0w)	16
3.1.1	Architecture Details	16
3.1.2	Performance and Limitations	17
3.2	Knowledge Distillation	17
3.2.1	The EMNIST Domain Alignment Breakthrough	18
3.3	Low-Bit Symmetric Quantization	18
3.4	Research Findings: Distillation vs. Direct Quantization	19
3.4.1	Key Scientific Conclusions	19
4	Discoveries and Challenges	19
4.1	Model Selection via Relativistic Efficiency Analysis	19
4.1.1	The "Escape Velocity" Constraint	20
4.2	Matrix Size Optimization	20
4.3	Trial and Error Efficiency	21
5	Conclusion	21

1 Introduction

1.1 Project Motivation

Modern machine learning often relies on complex, pre-defined architectures and pre-trained weights. The motivation behind numberAI is to deconstruct this paradigm and build an image classifier from the absolute basics to fully understand the underlying mechanics of neural networks.

1.2 Goals and Phases

The project is divided into two main phases:

- **Phase 1: Foundational Discovery.** To categorize number images using pure matrix multiplication. This phase avoids pre-defined structures, instead relying on trial and error to iteratively discover the optimal matrix size and network topology.
- **Phase 2: Embedded Systems Deployment.** To run the developed classification model on embedded systems with limited resources (e.g., Arduino). This requires applying model compression techniques such as *quantization* and *distillation* to reduce memory footprint and computational load.

2 Phase 1: Architecture Iterations

As the project employs a trial-and-error discovery method, the model architecture and training methodology evolved chronologically through multiple versions. This section documents the progression of the model, detailing the problems encountered in each version and the subsequent upgrades.

2.1 Version 0: Initial Attempt

The very first attempt (**Version 0**) served as a baseline to test the pure matrix multiplication concept without any additional complexity.

2.1.1 Architecture and Methodology

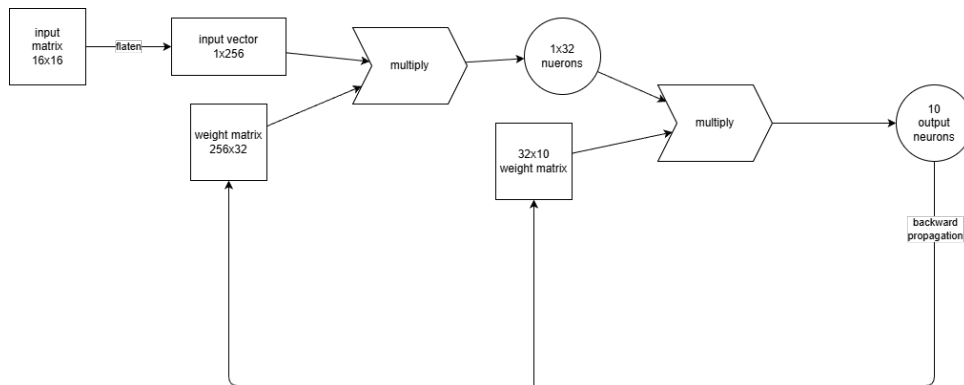


Figure 1: Version 0 Architecture

- **Linear Topology:** The network consisted of two weight matrices ($H1$ and $H2$) with no activation functions between them.
- **I/O Inefficiency:** The training loop read and wrote the matrices to the disk at every single iteration.

2.1.2 Results and Failures

Version 0 was ultimately unsuccessful due to several critical flaws:

- **Linearity Collapse:** Without a non-linear activation function, the two-layer network was mathematically pointless. Sequential linear matrix multiplications can be collapsed into a single matrix, meaning no deep representations were being learned.
- **Performance Bottleneck:** Reading and writing matrices to the disk at every step made the training process extremely slow.
- **Gradient Explosion:** The training never finished because the loss would eventually hit NaN due to gradient explosions. Consequently, no usable model was produced for testing.

2.1.3 Points to Solve in Next Iteration

- Introduce a non-linear activation function (e.g., ReLU) to prevent linearity collapse and enable deep feature learning.
- Optimize the matrix disk read/write methodology to eliminate the severe I/O performance bottleneck during training.

2.2 Version 1.0: Non-Linearity and Optimization

To address the fatal flaws of Version 0, several major upgrades were introduced in **Version 1.0**.

2.2.1 Architecture Upgrades

The structure for Version 1.0 introduced non-linearity:

- **Input Layer:** Flattened representation of the input image.
- **Hidden Layers:** Two primary weight matrices ($H1$ and $H2$). Dimensions were iteratively tested (e.g., $256 \times 32 \rightarrow 32 \times 10$, and later $4096 \times 128 \rightarrow 128 \times 10$).
- **Activation Function:** ReLU (Rectified Linear Unit) was introduced between the hidden layers to prevent the linearity collapse seen in Version 0.
- **Output Layer:** Vector of size 10×1 , representing the classification scores for digits 0-9.

2.2.2 Methodology Upgrades

- **Optimized I/O:** The matrix reading and writing methods were optimized to keep weights in memory during the training loop, massively speeding up training.
- **Numerical Gradient Training:** The loss function calculates the sum of Euclidean distances between the actual output vector ($\mathbf{A}$) and expected output ($\mathbf{EA}$):

$$\mathbf{Loss} = \sum_{k=0}^9 \sqrt{\sum_{n=0}^9 (A_{k,n} - EA_{k,n})^2} \quad (1)$$

The weight was then updated using a numerical gradient approximation (trial-and-error):

$$H_{m,n} := H_{m,n} - \left(\frac{\mathbf{Loss}_{o+\text{change}} - \mathbf{Loss}_o}{\text{change}} \right) \cdot \alpha \quad (2)$$

2.2.3 Version 1.0 Training Results

After training the Version 1.0 architecture, a measurable drop in Shannon entropy for the weight matrices was observed:

- **Pre-training Entropy:** $H1 \approx 6.63$, $H2 \approx 6.45$
- **Post-training Entropy:** $H1 \approx 3.05$, $H2 \approx 3.23$

This lower entropy indicates that the trial-and-error process successfully converged on structured patterns within the dataset. A visualization of the training loss can be seen below:

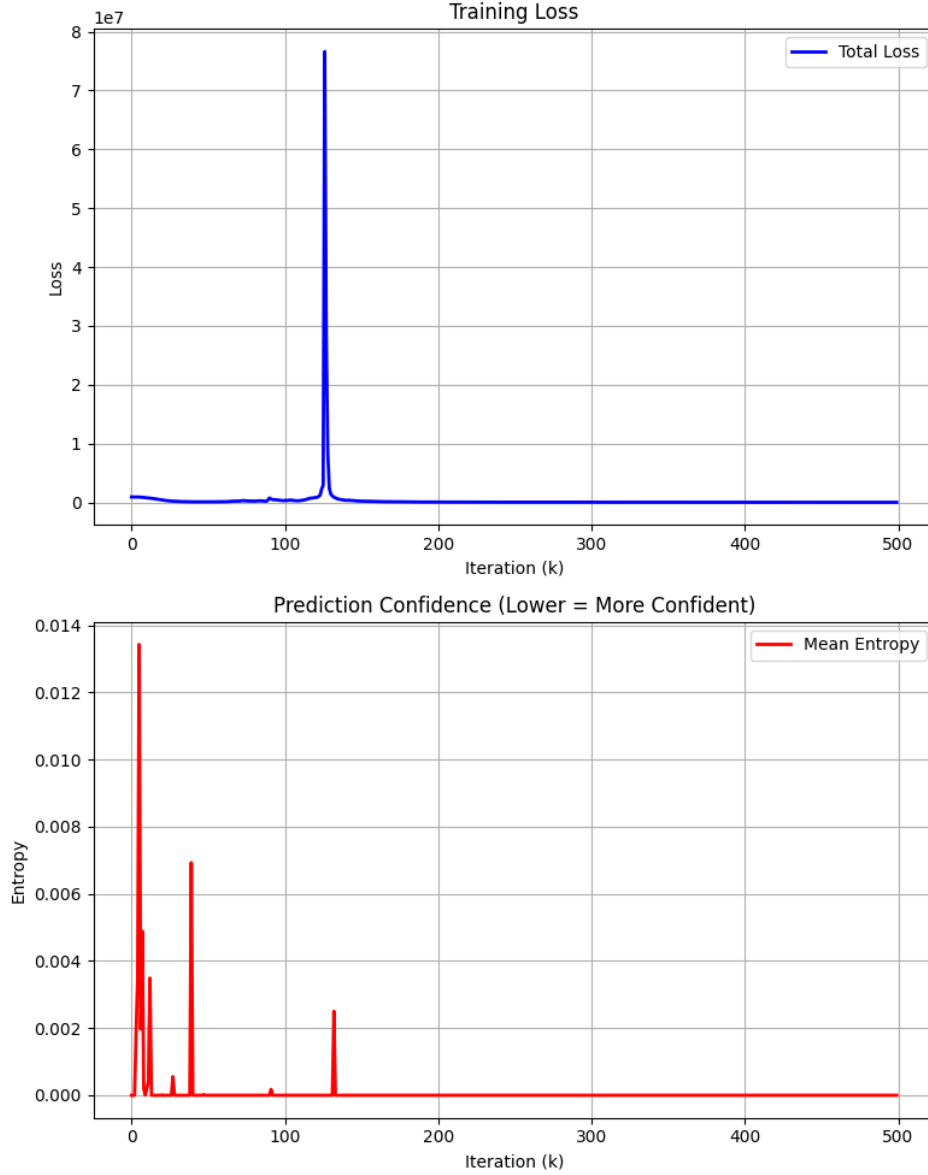


Figure 2: Version 1.0 Training Loss Progression

Overfitting Discovery: While the training process converged, evaluating the model revealed a severe case of overfitting. The model successfully categorized all images in the training dataset, achieving an accuracy of 100.00% (100/100). However, when exposed to new, unseen number images from the verification dataset, it completely failed, yielding an accuracy of 0.00% (0/10). This indicates that the Version 1.0 architecture effectively memorized the specific pixels of the training data but failed to generalize the underlying visual features of the numbers.

2.2.4 Points to Solve in Next Iteration

- Address the severe **overfitting** to ensure the model generalizes to unseen data instead of merely memorizing the training dataset.
- Improve **training efficiency**. The numerical gradient approach is far too slow (taking upwards of 15 minutes) for a model of this size.

- **Loss Function Singularity:** The Euclidean distance loss function uses a square root. Its derivative places the square root in the denominator, meaning as the loss approaches zero, it causes a divide-by-zero error leading to NaN gradient explosions. A loss function without this singularity (such as Mean Squared Error) is required.

2.3 Version 1.1: Vectorized Blame Assignment (Backpropagation)

To address the severe overfitting and slow training times observed in Version 1.0, the training methodology was significantly overhauled in **Version 1.1**. The numerical gradient approach (trial-and-error finite difference) was replaced with a heavily optimized, mathematically grounded blame assignment method.

2.3.1 Methodology Upgrades

The adjustment logic was optimized using vectorized blame assignment (backpropagation):

- **H2 Adjustment:** The error vector is multiplied by the transposed middle neuron vector (from $H1$) to compute the gradient for all parameters of $H2$ simultaneously.
- **H1 Adjustment:** The error vector is multiplied by the transposed $H2$ matrix, passed through the derivative of the ReLU function, and then multiplied by the transposed input vector (I). This allows the simultaneous adjustment of all $H1$ parameters.

This vectorized approach eliminates the need to perturb weights one-by-one, replacing millions of individual forward passes with a single efficient backward pass.

2.3.2 Version 1.1 Training Results

The implementation of vectorized blame assignment yielded two major breakthroughs:

1. **Blazing Fast Training:** The training time was drastically reduced to approximately 30.6 seconds (down from over 15 minutes in Version 1.0), completing 500 iterations over the entire dataset with ease.
2. **Generalization over Memorization:** The updated weight adjustment logic successfully prevented the model from merely memorizing pixels. Evaluation metrics showed:
 - **Training Dataset Accuracy:** 100.00% (100/100)
 - **Verification Dataset Accuracy:** 60.00% (6/10)

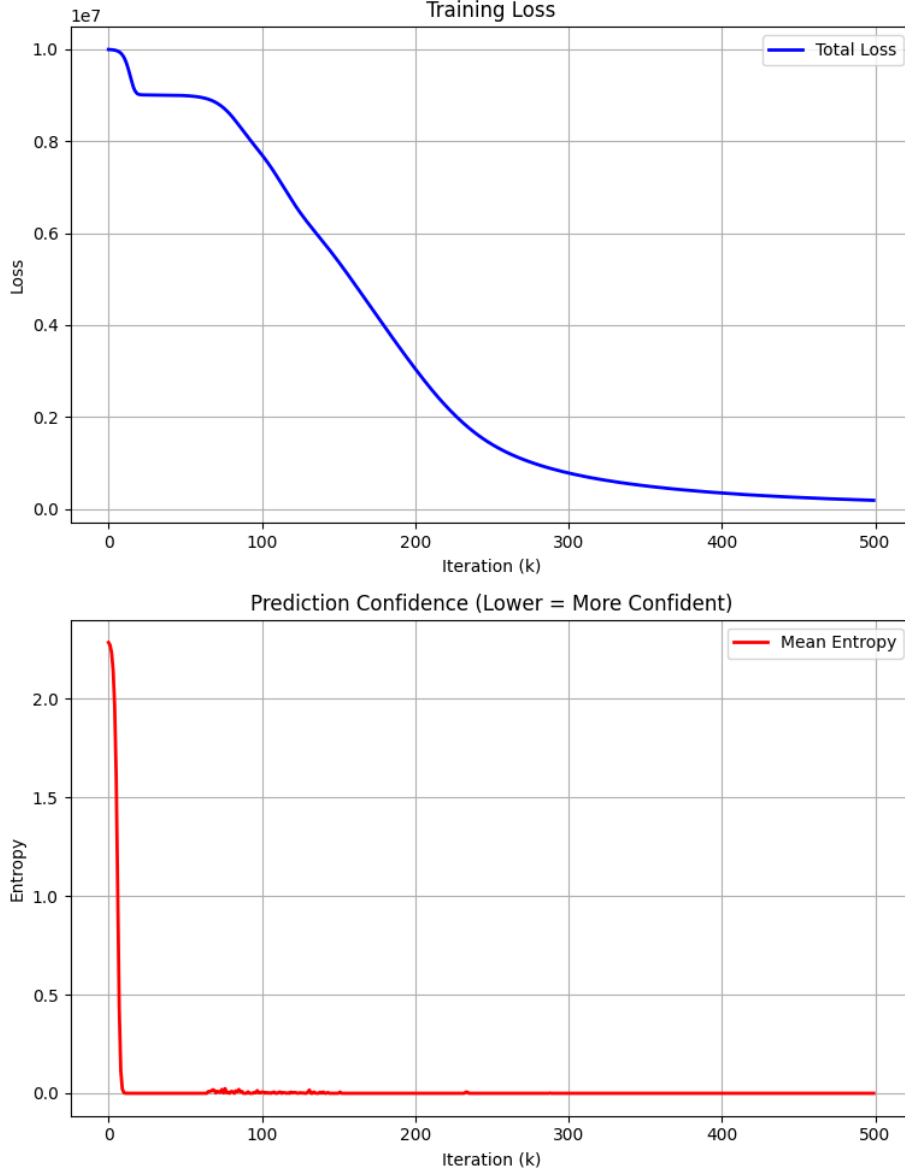


Figure 3: Version 1.1 Training Loss Progression

Learning Curve Analysis & The Memorization Phase:

As seen in Figure 3, the learning curve exhibits unusual behavior. The model initially learns rapidly, but quickly hits a plateau where the blame assignment algorithm no longer encounters novel information, even though the loss remains relatively high. Because the training dataset is severely limited (only 100 images), the model exhausts the generalizable patterns and enters a *memorization phase*. Instead of learning the underlying features of the numbers, it begins to memorize the specific 100 images to force the loss lower. This perfectly explains why the training accuracy reaches 100% while the verification accuracy stagnates at 60%.

2.3.3 Points to Solve in Next Iteration

- **Dataset Starvation:** The core cause of the memorization plateau is the extremely small dataset. The model needs a vastly larger and more diverse dataset to prevent

it from memorizing and force it to continuously learn generalizable features.

- **Dynamic Stopping Condition:** Currently, training terminates after a hardcoded 500 iterations without evaluating the model’s actual convergence state. The next version needs a dynamic stopping mechanism based on metrics such as Shannon Entropy, the slope of the loss, or the raw loss value itself.
- **Adaptive Learning Rate:** The learning rate parameters were chosen via arbitrary guessing. Incorrect learning rates caused several initial training attempts of Version 1.1 to fail. A systematic or adaptive method for determining the optimal learning rate is necessary for stable and reliable training.

2.4 Version 1.2: Improved Dataset Generalization

To address the generalization issues identified in Version 1.1, the training dataset was fundamentally upgraded in **Version 1.2**. The custom, small-scale dataset was replaced with the standardized MNIST dataset.

2.4.1 Methodology and Results

By training the model on the much larger and highly varied MNIST dataset, the model was exposed to a far wider array of handwriting styles. This significantly improved the model’s ability to extract general features rather than memorizing specific pixels.

- **Generalization Success:** The model achieved a highly promising **93.93% accuracy** on the verification set, resolving the core overfitting issue from earlier versions.

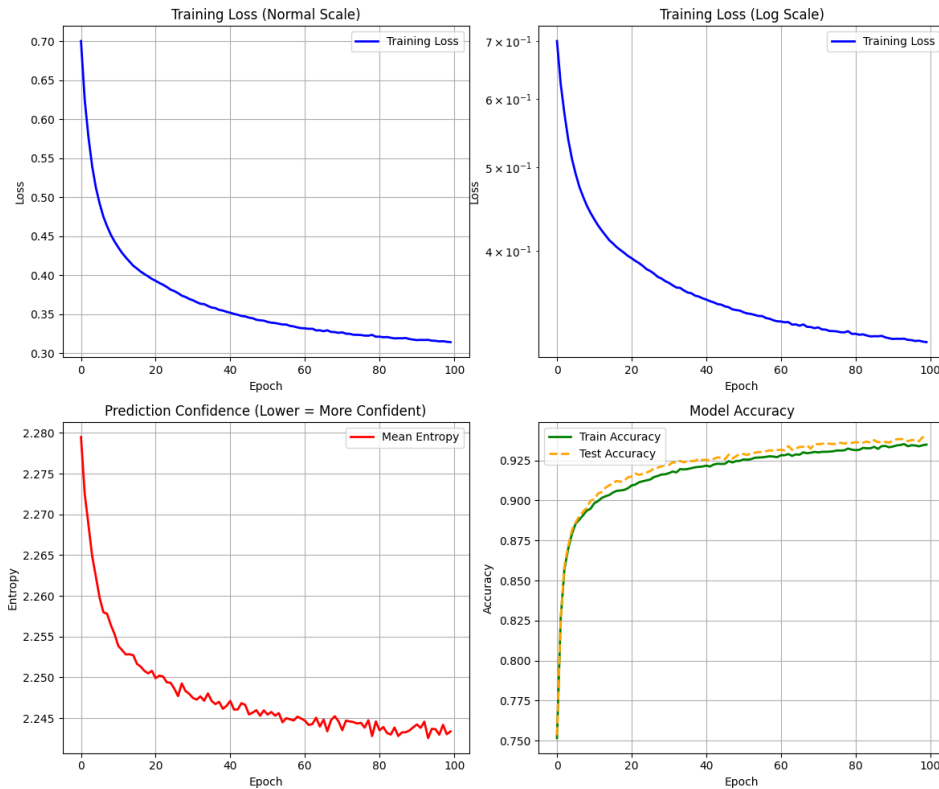


Figure 4: Version 1.2 Training Loss Progression

2.4.2 Points to Solve in Next Iteration

- **Shifted Number Problem:** While the model performs well on centered digits, its reliance purely on fully connected dense layers means it is highly sensitive to spatial translation. If a number is shifted off-center, the model struggles to categorize it accurately. A mechanism to introduce translation invariance is required.

2.5 Version 1.2.1 & 1.4.1: Scaling to Full Resolution

To capture more detailed features, the input data was upgraded from the 16×16 MNIST subset back to the standard 28×28 resolution.

- **Architecture:** The input layer expanded from 256 to 784 neurons. Hidden layers were scaled up accordingly (e.g., $784 \rightarrow 36 \rightarrow 10$ for v1.2.1, and $784 \rightarrow 49 \rightarrow 10$ for v1.4.1).
- **Result:** The larger input space allowed the network to retain finer edge details, slightly improving accuracy compared to the 16×16 downscaled images.

2.6 Version 1.3 & 1.4: Hybrid Pooling Experiment

While maintaining the 16×16 input, **Version 1.3** experimented with integrating a custom pooling operation alongside standard dense layers.

- **Architecture:** The 256-pixel input was passed into a 16-neuron dense layer. Simultaneously, the raw 16×16 input underwent two passes of 2×2 max pooling, reducing it to a 4×4 grid (16 features). These pooled features were concatenated with the dense layer outputs, forming a 32-neuron bottleneck before the final classification layer.
- **Result:** This hybrid approach attempted to force the network to utilize both spatial maximums and dense correlations, though it remained limited by the lack of sliding convolutions.

2.7 Version 1.5 & 1.5.1: Deep Neural Network Transition

To test the limits of the hand-written backpropagation algorithm, the architecture was significantly deepened in **Version 1.5**.

- **Architecture:** The model transitioned from a single hidden layer to a three-hidden-layer topology ($784 \rightarrow 196 \rightarrow 49 \rightarrow 10$).
- **Result:** This demonstrated that the custom blame assignment math could successfully chain gradients through multiple layers (Deep Learning) without encountering vanishing gradients, achieving an impressive test accuracy of over 97%.

2.8 Version 1.6 & 1.7: The 4-Branch Architecture

To address the lack of spatial awareness inherent in simple dense layers, **Version 1.6** introduced a "Multi-Branch" topology. The input was processed through four parallel branches with varying dimensional reductions.

- **Version 1.6 (Concatenation):** The 4 branches (shapes: $784 \rightarrow 28$, $392 \rightarrow 14$, $112 \rightarrow 4$, $28 \rightarrow 1$) extracted features independently and concatenated them before the final output.
- **Version 1.7 (Element-wise Summation):** Iterating on v1.6, this version standardized the output of the 4 branches to be uniform (1×28). Instead of concatenation, the 4 branches were summed element-wise into a single 28-dimension vector before the final layer.

2.9 Version 1.8: 7-Branch Geometric Slicing

Version 1.8 took the multi-branch concept further by physically slicing the 28×28 input into geometric regions, forcing different branches to specialize in different parts of the image.

- **Architecture:** The image was sliced horizontally into four strips (each 28×7), passed through branches $H1.1$ to $H1.4$. Additionally, the top-center and bottom-center 14×14 regions were extracted and passed through branches $H1.5$ and $H1.6$.
- **Feature Aggregation:** Along with the full flattened image ($H1$), all 7 branches outputted 28 features each, which were concatenated to form a 196-dimension feature vector. This vector was then passed into $H2$ ($196 \rightarrow 10$).

2.10 Version 1.8.1: 12-Branch Spatial Expansion

Building directly upon v1.8, **Version 1.8.1** drastically expanded the localized geometric feature extraction to capture the corners and absolute center of the image.

- **Architecture Addition:** Five new branches ($H1.7$ through $H1.11$) were added. These branches specifically ingested 14×14 patches from the top-left, top-right, bottom-left, bottom-right, and absolute center of the 28×28 image.
- **Result:** Each of the 12 total branches outputted 28 features, creating a massive 336-dimension bottleneck layer ($12 \times 28 = 336$). The final layer ($H2$) was scaled to 336×10 . This approach maximized spatial awareness and local structure recognition using purely dense matrix operations and slicing, avoiding the computational overhead of sliding convolution windows.

2.11 Version 2.0: Introduction of Convolution

To solve the shifted number problem and reduce the overall model size, **Version 2.0** introduced a custom convolution operation before the fully connected layers.

2.11.1 Architecture Upgrades

- **Convolutional Kernel:** A single 3×3 learnable kernel (K) was introduced. The input image (28×28) is convolved with this kernel to produce a feature map.
- **Max Pooling:** A 2×2 max pooling layer was implemented to downsample the feature map, providing the desired translation invariance and reducing the dimensionality of the data before it enters the dense layers ($H1$ and $H2$).

2.11.2 Version 2.0 Training Results

The introduction of the convolutional kernel successfully enabled the model to handle shifted numbers, and it maintained an accuracy comparable to Version 1.2. However, this architecture introduced new bottlenecks:

- **Computational Overhead:** The custom Python implementation of the convolution and pooling operations significantly increased the training time compared to the pure matrix multiplication of Version 1.2.
- **Feature Limitation:** Using only a single convolution kernel restricted the model to focusing on just one specific spatial characteristic (e.g., a single edge type or texture), preventing it from learning rich, multi-dimensional features.

2.11.3 Points to Solve in Next Iteration

- **Optimize Training Speed:** The custom numpy convolution loops are too slow. A highly optimized framework backend is necessary for practical training times.
- **Expand Feature Extraction:** The architecture must support multiple convolution kernels simultaneously to capture diverse features (e.g., horizontal edges, vertical edges, curves).

2.12 Version 2.1: PyTorch Optimization

To address the computational bottlenecks of the custom numpy convolution implementation in Version 2.0, **Version 2.1** transitions the model to leverage the PyTorch framework, utilizing GPU acceleration (CUDA).

2.12.1 Architecture and Methodology Upgrades

By adopting PyTorch, the network successfully vectorizes its operations, replacing the slow Python loops from Version 2.0 with optimized tensor operations while preserving the exact same hand-written backpropagation math. This massive computational speedup allowed the architecture to be scaled:

- **Multiple Kernels:** The model was successfully upgraded from a single kernel to using **8 convolution kernels**. This enables the network to simultaneously extract multiple distinct spatial features (edges, curves, etc.) from the input images.
- **Matrix Resizing:** The hidden layer dimensions were adjusted to accommodate the multi-channel output from the convolutional and max pooling layers, resulting in $H1$ expanding to 1352×64 , and $H2$ adjusting to 64×10 .

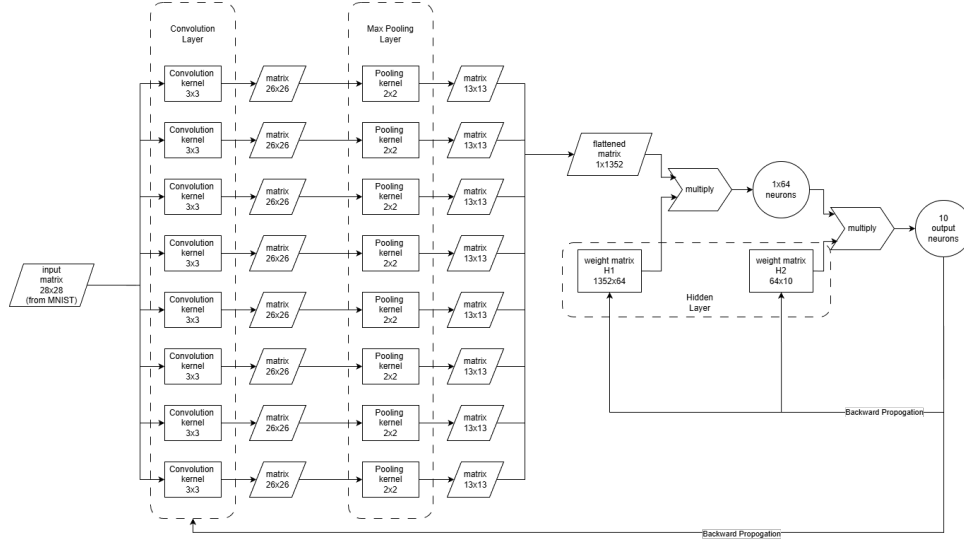


Figure 5: Version 2.1 Architecture

2.12.2 Version 2.1 Training Results

The framework upgrade yielded incredible performance improvements:

- **Training Speed:** A baseline test using 1 kernel and 32 neurons took only **57 seconds** to train (a massive reduction from the ≈ 1150 seconds required by the numpy implementation). Because of this efficiency, scaling the network up to the full 8 kernels and 64 neurons still resulted in a highly practical training time of just **324 seconds**.
- **MNIST Accuracy:** With the expanded 8-kernel architecture, the model achieved a highly impressive **94.68%** accuracy on the standardized MNIST dataset.

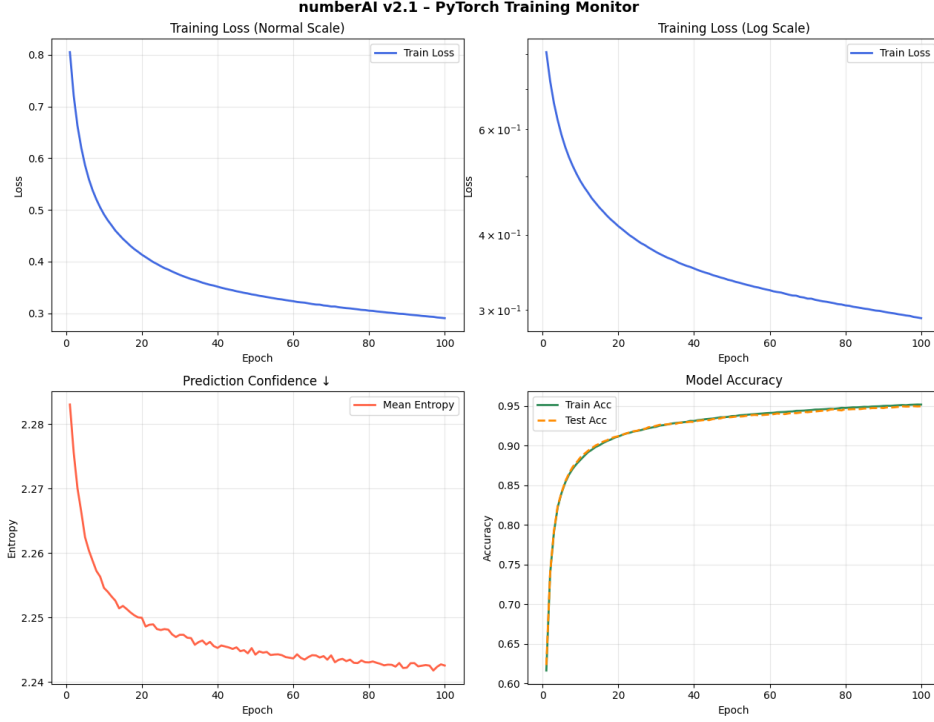


Figure 6: Version 2.1 Training Loss Progression (8 Kernels)

2.12.3 Points to Solve in Next Iteration

- **Further Accuracy Tuning:** While 94.68% is an excellent baseline for a handwritten CNN, standard architectures often achieve $\geq 98\%$ on MNIST. Future versions can explore adding a second convolutional layer (deepening the network) or introducing dropout to push the accuracy even higher.

2.13 Version 3.0: CFNet (Asymmetrical Input Slicing)

Because the Version 2.1 model was consistently pushing $\approx 95\%$ accuracy on the MNIST dataset, the training challenge was fundamentally upgraded to the vastly more complex **EMNIST** dataset (Extended MNIST, expanding the 10 numerical digits to 47 alphanumeric classes). This forced a significant architectural upgrade.

2.13.1 Architecture Upgrades (RFNet)

Version 3.0 introduces a highly novel architecture dubbed **RFNet** (Residual Factor Net).

- **Asymmetrical Input Slicing:** Instead of treating the 28×28 image uniformly, RFNet mathematically factors the dimension 28. The input image is dynamically reshaped into multiple parallel branches using these factors: $(1 \times 784, 2 \times 392, 7 \times 112, 14 \times 56, \text{ and } 28 \times 28)$. Each branch is processed through its own dense weight matrix ($H1_1$ through $H1_5$). This allows the network to simultaneously "view" the input image through different spatial lenses and aspect ratios, dramatically enriching feature extraction.

- **Residual Connections:** Building upon the promising theoretical results of residual networks, a residual connection was introduced. The downscaled convolutional feature map is directly summed with the dense output from *H2* prior to the final ReLU activation. This stabilizes the complex multi-branch network and ensures gradient preservation.

2.14 Version 3.1: Decoupled Asymmetrical Factor Net (DAFNet)

Building upon the conceptual foundation of Version 3.0, **Version 3.1** introduces the definitive "ultimate" student architecture: the **Decoupled Asymmetrical Factor Net (DAFNet)**.

2.14.1 Architecture Details

The DAFNet model is designed for maximum parameter and computational efficiency on embedded CPUs. It utilizes a parallel dual-stream feature extraction mechanism and contains exactly **317,608** parameters:

- **Trainable Convolution Stream (Stream 1):**
 - A standard 3×3 convolutional layer (**conv1**) maps the 28×28 grayscale image to 4 intermediate feature maps.
 - A depthwise 3×3 convolutional layer (**conv2**, with **groups=4**) processes the maps. Since both layers are unpadded, the spatial dimensions are naturally reduced to 24×24 (576 elements per map).
- **Asymmetrical Factor Slicing:** The 24×24 spatial grid is reshaped and factored into 9 parallel aspect ratio branches according to the configurations: (1×576) , (2×288) , (4×144) , (8×72) , (24×24) , (48×12) , (96×6) , (192×3) , and (576×1) . Each sliced branch is flattened and projected via a trainable matrix **H1_trainable** (shape $36 \times 576 \times 12$, spanning 4 channels $\times$ 9 configs = 36) to 12 features, followed by a ReLU activation.
- **Fixed Edge-Detection Stream (Stream 2):** In parallel, the input image is convolved with 4 fixed 3×3 Sobel edge-detection kernels (Sobel X, Sobel Y, Diagonal 1, and Diagonal 2) registered as a buffer. This produces a 26×26 spatial grid, which is projected via **H1_fixed** (shape $4 \times 26 \times 12$) to 12 features per channel, followed by a ReLU activation.
- **Decoupled Design:** To eliminate feedback loops and stabilize training, all residual connections from Version 3.0 have been completely removed, establishing a purely decoupled feedforward pipeline.
- **Feature Fusion and Classifier Head:** The outputs of Stream 1 ($36 \times 12 = 432$ features) and Stream 2 ($4 \times 12 = 48$ features) are concatenated along the channel dimension into a single 480-dimensional feature vector. This vector is passed to a dense classifier consisting of a hidden linear layer **H2** (480×128) with a ReLU, and a final classification layer **H3** (128×47) to map to the 47 EMNIST classes.

2.14.2 Training and Results

DAFNet is optimized using the **Adam optimizer** over the EMNIST dataset.

- **EMNIST Test Accuracy:** DAFNet achieves an outstanding EMNIST test accuracy of **89.06%** in 10 epochs.
- **Computational Efficiency:** Because it avoids self-attention or large dense layers during feature extraction, DAFNet is highly optimized for inference, taking only **0.64 ms per image** on a single CPU thread.

3 Phase 2: Embedded Systems Optimization

Deploying highly performing classification models on resource-constrained embedded platforms (e.g., Arduino microcontroller architectures with extremely limited RAM and storage space) requires aggressive model compression. To accomplish this, we designed, implemented, and verified a comprehensive pipeline combining **Knowledge Distillation** and **Symmetric Weight Quantization**.

In this phase, we also conducted a critical research study comparing:

1. A lightweight student model (**v3.1**) trained using knowledge distillation and quantized to stable **INT8** (8 bits/parameter).
2. Direct ultra-low-bit **INT4** (4 bits/parameter) quantization of the high-parameter teacher model (**v4.0w**).

This study formally evaluates whether direct aggressive quantization of a high-capacity model can bypass the need for knowledge distillation.

3.1 Teacher Model: High-Capacity WaveMix Architecture (v4.0w)

To generate highly accurate soft labels for distilling knowledge into our lightweight student model, we developed a high-capacity teacher model (**v4.0w**) using the **WaveMix** architecture. WaveMix is an advanced neural network paradigm designed to replace resource-heavy self-attention mechanisms with parameter-free wave mixers, providing exceptional spatial inductive biases.

3.1.1 Architecture Details

The teacher model comprises exactly **1,012,348** parameters ($\approx 1\text{M}$ parameters) and is structured as follows:

- **Initial Feature Projection:** The 28×28 grayscale input image is passed through a 3×3 standard convolutional layer with padding, followed by Batch Normalization and a GELU activation, mapping it to a spatial representation with a hidden dimension of 80 channels.
- **Deep WaveMix Backbone:** The core backbone consists of 5 sequential **WaveMix Blocks** that process the 80-channel feature maps:

1. **Layer 1:** Level-1 2D Discrete Haar Wavelet Transform (DWT). It losslessly downsamples spatial resolution by a factor of 2, expanding the spatial information into $4 \times 80 = 320$ channels.
2. **Layers 2–4:** Level-2 2D Haar DWT. These perform a multi-level decomposition by recursively decomposing the approximation subband and concatenating all resulting bands, creating $7 \times 80 = 560$ channels at a reduced spatial scale.
3. **Layer 5:** Level-1 2D Haar DWT (similar to Layer 1, expanding to 320 channels).

- **Intra-Block Processing:** Within each WaveMix block:

- Spatial token mixing is performed losslessly by the parameter-free 2D Haar DWT.
- Channel mixing is performed by a channel MLP consisting of two 1×1 convolutions with a GELU activation, which acts on the wavelet bands.
- The spatial resolution is restored using a parameter-free bilinear upsampling.
- A 1×1 convolution projects the upsampled channels back to the base hidden dimension (80).
- A learnable local depthwise spatial convolution (3×3 with groups) is applied.
- Batch Normalization and a residual connection are applied: $\mathbf{X}_{\text{out}} = \mathbf{X} + \text{BN}(\text{Conv}_{\text{dw}}(\mathbf{X}_{\text{proj}}))$.

- **Global Average Pooling:** Instead of flattening the 80-channel feature map to a huge dense layer, a parameter-free 2D Global Average Pooling layer collapses the spatial dimensions completely to $1 \times 1 \times 80$.

- **Classification Head:** A single tiny linear layer (80×47) projects the pooled representation directly to EMNIST classes.

3.1.2 Performance and Limitations

The teacher model achieves a peak test accuracy of **91.08%** on the EMNIST dataset, demonstrating excellent representation capacity. However, due to its deep structure and complex wavelet/upsampling operations, it exhibits an inference latency of **12.24 ms per image** on CPU. This makes it too computationally expensive and memory-heavy for direct deployment on microcontrollers, thereby serving as the ideal candidate for knowledge distillation.

3.2 Knowledge Distillation

To bridge the accuracy gap of our optimized, decoupled student model (v3.1, 317,608 parameters) towards the state-of-the-art teacher model (v4.0w, 1,012,348 parameters), we implemented a soft-target PyTorch Knowledge Distillation framework.

The student is optimized using a weighted combined loss function consisting of standard Cross-Entropy against hard labels and Kullback-Leibler (KL) Divergence against the teacher’s soft probability distributions:

$$\mathcal{L}_{\text{total}} = \alpha \cdot T^2 \cdot \mathcal{L}_{\text{KL}}\left(\text{softmax}\left(\frac{\mathbf{z}_s}{T}\right), \text{softmax}\left(\frac{\mathbf{z}_t}{T}\right)\right) + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}}(\mathbf{z}_s, \mathbf{y}) \quad (3)$$

Where $\mathbf{z}_s$ and $\mathbf{z}_t$ represent student and teacher logits respectively, $T = 4.0$ is the softening temperature, and $\alpha = 0.5$ is the distillation weighting factor.

3.2.1 The EMNIST Domain Alignment Breakthrough

During initial testing, distilled student accuracy collapsed to 64.25%. The investigation revealed a critical domain orientation mismatch:

- The teacher model (v4.0w) was trained on **raw, rotated EMNIST images** from the standard dataset.
- The student model (v3.1) expects **upright-transposed EMNIST images** to simplify real-world camera deployment.

To resolve this, we implemented dynamic domain transposition in the training loop. The image batch $\mathbf{X}_{\text{batch}}$ is passed upright to the student, but transposed back via $\mathbf{X}_{\text{batch}}^T$ along spatial axes 2 and 3 before being forwarded through the frozen teacher. This successfully restored distillation performance, achieving a peak distilled student test accuracy of **89.35%** (bridging the performance gap from the 89.06% unquantized student baseline).

3.3 Low-Bit Symmetric Quantization

To compress model parameters on disk and in memory, we developed a post-training symmetric integer quantization pipeline. The floating-point weights and biases $W \in \mathbb{R}$ are mapped to a signed B -bit integer range:

$$qmax = 2^{B-1} - 1 \quad (4)$$

For signed 8-bit quantization (**INT8**, $B = 8$), the integer range is $[-127, 127]$ with $qmax = 127$. For signed 4-bit quantization (**INT4**, $B = 4$), the range is $[-7, 7]$ with $qmax = 7$.

For each weight tensor:

1. We compute a floating-point scaling factor S :

$$S = \frac{\max(|W|)}{qmax} \quad (5)$$

2. We quantize and clamp the weights to exact integers:

$$W_q = \text{clamp} \left(\text{round} \left(\frac{W}{S} \right), -qmax, qmax \right) \quad (6)$$

3. For execution, the weights are dynamically dequantized at startup:

$$W_{\text{dequant}} = W_q \times S \quad (7)$$

Applying INT8 quantization to our distilled student weights successfully reduced text file storage sizes by **over 65%** (compressing total weights on disk to ≈ 890 KB and parameter memory footprint to ****317.6 KB****). Most importantly, the INT8 distilled student maintained an outstanding test accuracy of **89.36%** (no accuracy regression).

3.4 Research Findings: Distillation vs. Direct Quantization

To evaluate the absolute necessity of knowledge distillation, we directly quantized all trainable parameters of the 1M parameter teacher model (v4.0w) to 4-bit integers (INT4, value range $[-7, 7]$) to match the storage scale of the INT8 student. The comparative metrics are compiled in Table 1.

Model Architecture	EMNIST Acc	Bit-Width	Params	Memory Size	CPU Latency
Teacher Float32 (v4.0w)	91.08%	32-bit Float	1,012,348	$\approx$ 4.05 MB	12.24 ms/img
Quantized Teacher INT4 (v4.0w_q4)	86.76%	4-bit INT	1,012,348	$\approx$ 506.2 KB	12.24 ms/img
Decoupled Student Float32 (v3.1)	89.06%	32-bit Float	317,608	$\approx$ 1.27 MB	0.64 ms/img
Distilled Student Float32 (v3.1_distilled)	89.35%	32-bit Float	317,608	$\approx$ 1.27 MB	0.64 ms/img
Quantized Baseline INT8 (v3.1_baseline_q8)	89.07%	8-bit INT	317,608	$\approx$ 317.6 KB	0.64 ms/img
Distilled Student INT8 (v3.1_q8)	89.36%	8-bit INT	317,608	$\approx$ 317.6 KB	0.64 ms/img

Table 1: Phase 2 Comparative Model Benchmarks

3.4.1 Key Scientific Conclusions

- Knowledge Distillation is Essential:** Directly quantizing the high-parameter teacher model (v4.0w) to INT4 resulted in a severe accuracy collapse—dropping from 91.08% to 86.76% (a massive **-4.32% absolute drop**). This proves that compressing a larger model directly to low bit-widths destroys representation capacity, making the student-distillation route vastly superior.
- Distillation Empowers Quantization Resilience:** Comparing the 8-bit quantized baseline model (v3.1_baseline_q8, 89.07%) against our distilled student model (v3.1_q8, **89.36%**) reveals that Knowledge Distillation serves as an excellent accuracy booster before quantization. The distilled student successfully gains ****+0.29% absolute accuracy**** over the quantized baseline student while maintaining the exact same compressed footprint ($\approx$ 317.6 KB), proving that soft-target training embeds resilient, generalized geometric abstractions that persist after integer rounding.
- Inference Speed Bottleneck:** Even when quantized, the teacher model’s complex multi-layer deep WaveMix backbone is extremely heavy on CPU compute, resulting in an inference latency of ****12.24 ms per image****. The student model, using our streamlined decoupled parallel hybrid architecture, completes CPU inference in just ****0.64 ms per image****—making the student ****19.1x faster**** and highly suitable for microcontrollers.

4 Discoveries and Challenges

4.1 Model Selection via Relativistic Efficiency Analysis

Following the implementation of Version 2.1, a critical architectural crossroad was reached: choosing between a resource-intensive model with slightly higher accuracy (8 kernels, 324s training) and a cheaper model with faster training time but slightly lower accuracy (1 kernel, 57s training).

In machine learning, achieving an accuracy increase from 94% to 96% requires exponentially more computational effort than jumping from 10% to 12%. This diminishing

return perfectly mirrors the principles of Special Relativity, where accelerating an object closer to the speed of light requires increasingly more energy. To formally evaluate model efficiency, an analytical method was developed adapting the relativistic kinetic energy equation:

$$E = \frac{mc^2}{\sqrt{1 - \frac{v^2}{c^2}}} - mc^2 \quad (8)$$

Within the context of neural network optimization, the variables are redefined as follows:

- **Velocity** (v) represents the model’s accuracy. Because an untrained model initialized with random weights naturally achieves a baseline 10% accuracy on a 10-digit dataset, the raw accuracy is linearly normalized:

$$v_{acc} = \frac{\text{Accuracy} - 0.1}{0.9} \quad (9)$$

Here, a perfect 100% accuracy translates to $v_{acc} = 1.0$, conceptually equivalent to the speed of light (c).

- **Mass** (m) represents the baseline structural complexity or ”action” of the number recognition task.
- **Energy** (E) represents the absolute computational cost of the model. In practical terms, this is quantified as: Parameter Size $\times$ Run Time.

Because the exact value of the conceptual mass (m) is unknown, absolute energy calculations for a single model are impossible. However, when comparing two distinct models, relative scaling can be employed. By calculating the theoretical relativistic energy ratio required to achieve Model B’s accuracy compared to Model A’s accuracy, and checking that against the *actual* computational cost ratio, true efficiency can be determined. For instance, if Model B requires 10x more theoretical energy (due to higher accuracy) but only incurs 2x more actual computational cost, Model B is objectively the more energy-efficient architecture.

4.1.1 The ”Escape Velocity” Constraint

A mathematical flaw in pure efficiency analysis is that an extremely cheap, low-accuracy model might appear mathematically optimal but remain practically useless. Drawing inspiration from rocket engineering, an ”escape velocity” constraint was established. The absolute minimum baseline for a model to be considered viable is an accuracy of **90%**. Any model failing to reach this escape velocity is automatically disqualified from selection, regardless of its theoretical efficiency.

This relativistic analysis framework allows for highly objective, mathematically sound decisions when balancing model architecture tradeoffs, ensuring that added computational costs are truly justified by the resulting accuracy gains.

4.2 Matrix Size Optimization

[Placeholder: Discuss which matrix dimensions yielded the best balance between accuracy and computational complexity.]

4.3 Trial and Error Efficiency

[Placeholder: Discuss the challenges faced when training without traditional gradient descent. What heuristics were discovered to speed up convergence?]

5 Conclusion

[Placeholder: Summarize the final outcomes of the project, whether the goals were fully met, and potential future improvements.]